

Technical Report

Master's Programme in Data Science

# Priced Right? Estimating asking prices of products on Tori.fi

Abtaal Aatif, Hashir Salam Awan and Hafiz Khurram Javid

October 13, 2026

Faculty of Science

University of Helsinki

**Contact information**

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki, Finland

URL: https://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

| Tiedekunta — Fakultet — Faculty | Koulutusohjelma — Utbildningsprogram — Study programme |
|---|---|
| Faculty of Science | Master's Programme in Data Science |

Tekijä — Författare — Author

Abtaal Aatif, Hashir Salam Awan and Hafiz Khurram Javid

Työn nimi — Arbetets titel — Title

Priced Right? Estimating asking prices of products on Tori.fi

Ohjaaja(t) — Handledare — Supervisor(s)

| Työn laji — Arbetets art — Level | Aika — Datum — Date | Sivumäärä — Sidoantal — Number of pages |
|---|---|---|
| Technical Report | October 13, 2026 | 6 pages |

Tiivistelmä — Sammanfattning — Abstract

Second-hand sellers on Tori.fi guess their asking prices, and buyers cannot easily tell whether a listing is fair. We cleaned 2,178 collected iPhone listings to 2,007, trained five regression models on a grouped development set, and chose gradient boosting with rules fixed in advance. On 405 held-out listings it reached an RMSLE of 0.272, 27% below a lookup baseline, with a mean absolute error of 46 euros. Calibrated 80% ranges covered 77% of test listings. A Flask application serves sellers and buyers. Estimates describe asking prices, not sale prices, and are weakest for phones under 100 euros.

**ACM Computing Classification System (CCS)**
Computing methodologies → Machine learning → Learning paradigms → Supervised learning → Supervised learning by regression

Avainsanat — Nyckelord — Keywords

price prediction, gradient boosting, second-hand marketplace, conformal prediction

Säilytyspaikka — Förvaringsställe — Where deposited

Muita tietoja — Övriga uppgifter — Additional information

# Contents

**1 Introduction** **1**

**2 From listings to an application** **2**

2.1 Data collection and cleaning . . . . . 2

2.2 Model selection . . . . . 3

2.3 Held-out results and price ranges . . . . . 3

2.4 The application . . . . . 4

**3 Conclusions** **5**

**References** **6**

# Chapter 1

# Introduction

Sellers on Tori.fi, a major Finnish second-hand marketplace, have to guess an asking price. A price that is too high leaves a listing without replies for weeks, while a price that is too low gives money away. Buyers face the opposite problem: to judge whether a listing is fair, they would have to compare dozens of similar advertisements by hand. Thousands of listings contain the signal both groups need, but no individual user can aggregate it efficiently.

Priced Right? turns these listings into a decision aid for second-hand iPhones. In *seller mode*, a user describes a phone and receives an expected asking price with an 80% range. In *buyer mode*, the user also enters a listing's price and learns whether it is below, in line with or above comparable listings. Because Tori never publishes final sale prices, every estimate describes what sellers *ask*, not what phones sell for, and a price above the range is unusual rather than wrong.

The project canvas originally covered phones and furniture, with bicycles as a stretch goal. We narrowed the scope to iPhones because Tori records their model, storage and graded condition as structured attributes, which makes the notion of a comparable listing precise. Collecting one category thoroughly also allowed more careful cleaning and evaluation than three categories collected thinly. The approach itself is general: any Tori category whose listings carry structured attributes could be added in the same way, and iPhones are its first application.

Chapter 2 describes the data collection and cleaning, the model selection, the held-out results with their price ranges, and the web application. Chapter 3 discusses limitations, future work and what we learned.

# Chapter 2

# From listings to an application

This chapter follows the project from raw listings to a deployed tool: the data, the selection of a model under rules fixed in advance, its single evaluation on held-out listings, and the application built on it.

## 2.1 Data collection and cleaning

On 17 September 2026 a purpose-built crawler collected 2,178 for-sale iPhone listings from Tori. It fetched only paths allowed by `robots.txt` and sent at most one request every three seconds. Data were minimised at collection time: seller identifiers, coordinates, postal codes and photos were never stored, and contact details were scrubbed from titles and descriptions. An audit found eleven contact details the first scrubber had missed, so we extended it and re-scrubbed the data; because descriptions may still name people, the repository is private. The canvas planned to use Tori only with written permission and the eBay Browse API otherwise; collection instead proceeded with the course supervisor's approval, a deviation we record here.

Cleaning removed accessories, spare parts, placeholder prices, multi-phone bundles and one Samsung phone, leaving 2,007 listings. We derived the exact model (51 models from 2008 to 2026), release year, storage (96% of listings), battery health (51%) and negation-aware text flags such as mentioned damage. Reposts of the same advertisement were grouped by their text, and 20% of the listings (405), grouped in the same way, were held out as a test set used only once. On the 1,602 development listings, asking prices were right-skewed (skewness 1.57, against $-0.46$ for log prices). Relative to the median price

of the same model and storage, retailers asked about 22% more, phones needing repair about 60% less and listings mentioning damage about 22% less.

## 2.2 Model selection

Because effects such as these are multiplicative, all models were trained on $\log(1 + \text{price})$ and compared by the root mean squared logarithmic error (RMSLE); we also report the mean absolute error (MAE). Every score came from the same five-fold grouped cross-validation, with all preprocessing inside scikit-learn pipelines on training folds only (Pedregosa et al., 2011). Inputs were limited to what a user knows before posting an advertisement. A feature check kept the phone model, condition, storage, seller type and mentioned damage, and dropped battery, region, shipping and the other text flags, which changed the RMSLE by 0.002 or less. Five tuned model families were compared with a lookup baseline, the median price of the same model and storage (Table 2.1).

| Model | RMSLE | MAE (euros) |
|---|---|---|
| Lookup baseline | $0.369 \pm 0.022$ | $56 \pm 4$ |
| Linear regression | $0.312 \pm 0.031$ | $53 \pm 3$ |
| Decision tree | $0.308 \pm 0.017$ | $51 \pm 3$ |
| Random forest | $0.293 \pm 0.016$ | $49 \pm 5$ |
| Support vector regression | $0.292 \pm 0.016$ | $47 \pm 4$ |
| Gradient boosting | $0.284 \pm 0.015$ | $47 \pm 3$ |

Table 2.1: Cross-validated scores on the 1,602 development listings (mean ± standard deviation over five grouped folds).

Gradient boosting (Friedman, 2001), with 400 rounds of four-leaf trees, had the lowest RMSLE and was the only model within one standard error, the rule set in advance. Its grid was widened three times, giving it more tuning than the others, and two models were within about 0.01, so the choice is close.

## 2.3 Held-out results and price ranges

Scored once on the 405 test listings, the model's RMSLE was 27% below the baseline's (95% CI 19–35%), with confidence intervals from 2,000 bootstrap resamples by advertisement group (Table 2.2; Efron and Tibshirani, 1993). All success criteria set before modelling were met.

| Metric | Final model | Lookup baseline |
|---|---|---|
| RMSLE | 0.272 (0.242–0.302) | 0.374 (0.331–0.418) |
| MAE (euros) | 46 (42–52) | 58 (51–64) |
| Median percentage error | 14.3% | 16.5% |
| 80% range coverage | 77% | – |

Table 2.2: Results on the 405 held-out test listings, with 95% bootstrap confidence intervals.

Because one number hides uncertainty, each estimate comes with an 80% range: a conformal interval built from out-of-fold errors, with a separate width per launch period in the spirit of Mondrian conformal prediction (Vovk et al., 2005). Old phones, which are harder to price, get wider ranges (a factor of 1.86 for 2016 or earlier against 1.19 for 2023 or later). The ranges covered 77% of test listings, 78% for private sellers and 75% for retailers. Errors were uneven, with a median of 7% for phones from 2023 or later but 27% for 2016 or earlier; the weakest group was the 82 listings asking 100 euros or less, which include broken phones listed as working, where estimates were typically 28% too high and the ranges covered only 65%.

## 2.4 The application

After the test, the model was retrained on all 2,007 listings and served by a Flask application at https://priced-right.onrender.com that offers both modes (Figure 2.1). It validates every input on the server, draws the range as a bar, flags rough estimates and stores nothing a user enters.

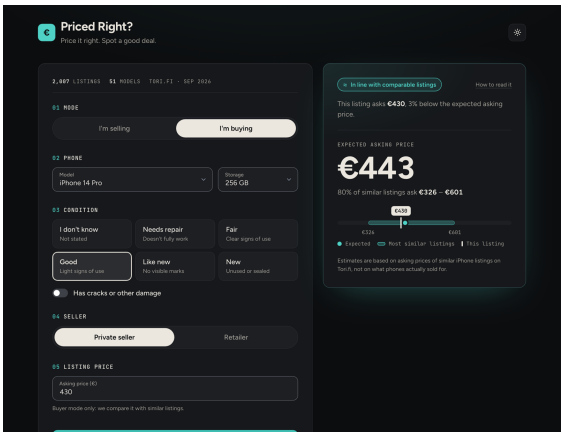



Figure 2.1: The application in buyer mode: a listing at 430 euros is in line with comparable listings.

# Chapter 3

# Conclusions

Priced Right? shows that a few structured attributes of an iPhone listing explain most of its asking price. Gradient boosting on launch year, product line, condition, storage, seller type and mentioned damage reduced the error of a model-and-storage lookup by 27% on held-out listings, and calibrated ranges communicate the remaining uncertainty honestly.

The results have clear limits. The model learns what sellers ask, not what buyers pay, because Tori does not publish sale prices. The data are one snapshot of one marketplace on 17 September 2026, so we cannot tell which listings sold or how long they stayed online, and seasonal effects are not captured. The model is least reliable for phones under about 100 euros, where estimates are too high and the ranges miss a third of listings. Reworded reposts may escape duplicate detection, and the project covers iPhones only rather than the furniture and bicycles in the canvas.

Future work follows from these limits. Repeated collections would show which listings disappear, giving a proxy for selling prices and a fresh test set, and the share of listings inside their range would serve as a drift check. The model should be retrained each September when Apple releases new phones, and quantile regression or a separate model could improve the estimates for cheap and broken phones.

We also learned that data collection is an engineering task in its own right: respecting robots rules, parsing embedded JSON and auditing for personal data took as much care as the modelling. Cleaning mattered as much as the choice of algorithm, since accessories, spare parts and other brands hidden among the listings would otherwise have distorted the model. Fixing the evaluation rules before modelling and opening the test set once made the final numbers trustworthy, and from the modelling stage on, every change went through a reviewed pull request with automated tests.

# References


Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall. DOI: 10.1201/9780429246593.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, *29*(5), 1189–1232. DOI: 10.1214/aos/1013203451.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html.

Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic learning in a random world*. Springer. DOI: 10.1007/b106715.